

Session 6: Grouping and SubqueriesTrainer Coverage:GROUP BY, HAVING.Subqueries and nested queries.Example: “Show users whose average daily spend > ?500.”Pending

Tasks

- 1) Create a SQL query using GROUP BY to show the total number of orders placed by each user in an 'orders' table, displaying user_id and order_count.

QUERY:

```
1 CREATE TABLE orders (  
2     order_id INT PRIMARY KEY AUTO_INCREMENT,  
3     user_id INT NOT NULL,  
4     order_date DATE  
5 );  
6  
7 INSERT INTO orders (user_id, order_date) VALUES  
8 (1, '2026-08-01'),  
9 (1, '2026-08-05'),  
10 (1, '2026-08-10'),  
11 (2, '2026-08-02'),  
12 (2, '2026-08-07'),  
13 (3, '2026-08-03'),  
14 (3, '2026-08-08'),  
15 (3, '2026-08-15');
```

```
16  
17 SELECT  
18     user_id,  
19     COUNT(*) AS order_count  
20 FROM orders  
21 GROUP BY user_id;
```

OUTPUT:

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create \]](#)

☐ Show all | Number of rows: 25

user_id	order_count
1	3
2	2
3	3

2) Write a SQL query to display the names of restaurants from a 'restaurants' table that have received an average rating above 4.0, using GROUP BY and HAVING.

QUERY:

Run SQL query/queries on table my database.restaurant:

```
1 SELECT name, AVG(rating) AS average_rating
2 FROM restaurant
3 GROUP BY name
4 HAVING AVG(rating) > 4.0;
```

OUTPUT:

Extra options	
←T→	▼ name average_rating
☐ Edit Copy Delete	Dragon Wok 4.600000
☐ Edit Copy Delete	Spice Garden 4.700000
☐ Edit Copy Delete	Sushi World 4.700000

3) Suppose you have a 'payments' table with columns user_id, amount, and payment_date. Write a subquery to find user_ids who have made a single payment above ₹2000 in any transaction.

QUERY:

Run SQL query/queries on database my database: 

```
1 CREATE TABLE payments (  
2     payment_id INT PRIMARY KEY AUTO_INCREMENT,  
3     user_id INT NOT NULL,  
4     amount DECIMAL(10,2) NOT NULL,  
5     payment_date DATE  
6 );  
7  
8 INSERT INTO payments (user_id, amount, payment_date) VALUES  
9 (1, 1500.00, '2026-08-01'),  
10 (1, 2500.00, '2026-08-05'),  
11 (2, 1800.00, '2026-08-03'),  
12 (2, 3000.00, '2026-08-07'),  
13 (3, 1200.00, '2026-08-04'),  
14 (3, 1900.00, '2026-08-08'),  
15 (4, 5000.00, '2026-08-10');
```

```
1 SELECT DISTINCT user_id  
2 FROM payments  
3 WHERE amount > 2000;
```

OUTPUT:

Extra options

←T→	payment_id	user_id	amount	payment_date
☐  Edit  Copy  Delete	1	1	1500.00	2026-08-01
☐  Edit  Copy  Delete	2	1	2500.00	2026-08-05
☐  Edit  Copy  Delete	3	2	1800.00	2026-08-03
☐  Edit  Copy  Delete	4	2	3000.00	2026-08-07
☐  Edit  Copy  Delete	5	3	1200.00	2026-08-04
☐  Edit  Copy  Delete	6	3	1900.00	2026-08-08
☐  Edit  Copy  Delete	7	4	5000.00	2026-08-10

☐ Show all | Number of rows: 25

▼

 | Filter

Extra options

←T→	user_id
☐  Edit  Copy  Delete	1
☐  Edit  Copy  Delete	2
☐  Edit  Copy  Delete	4

- 4) Using a nested subquery, display the names of movies from a 'movies' table that have a higher average rating than the overall average rating of all movies.

Hint: Use a subquery to calculate the overall average rating, then compare each movie's average rating against it in the main query.

QUERY:

```
1 CREATE TABLE movies (  
2     movie_id INT PRIMARY KEY AUTO_INCREMENT,  
3     name VARCHAR(100) NOT NULL,  
4     rating DECIMAL(3,1) NOT NULL  
5 );  
6  
7 INSERT INTO movies (name, rating) VALUES  
8 ('Avengers', 4.5),  
9 ('Avengers', 5.0),  
10 ('Titanic', 4.0),  
11 ('Titanic', 3.5),  
12 ('Inception', 4.8),  
13 ('Inception', 4.5),  
14 ('Avatar', 3.5),  
15 ('Avatar', 4.0),
```

```
1 SELECT name, AVG(rating) AS average_rating  
2 FROM movies  
3 GROUP BY name  
4 HAVING AVG(rating) > (  
5     SELECT AVG(rating)  
6     FROM movies  
7 );|
```

an SQL query, queries on table comp

```
1 SELECT AVG(rating)  
2 FROM movies;|
```

OUTPUT:

Extra options

				movie_id	name	rating
☐		Edit		Copy		Delete
				1	Avengers	4.5
☐		Edit		Copy		Delete
				2	Avengers	5.0
☐		Edit		Copy		Delete
				3	Titanic	4.0
☐		Edit		Copy		Delete
				4	Titanic	3.5
☐		Edit		Copy		Delete
				5	Inception	4.8
☐		Edit		Copy		Delete
				6	Inception	4.5
☐		Edit		Copy		Delete
				7	Avatar	3.5
☐		Edit		Copy		Delete
				8	Avatar	4.0
☐		Edit		Copy		Delete
				9	Interstellar	5.0
☐		Edit		Copy		Delete
				10	Interstellar	4.5

☐ Show all | Number of rows: 25 Filter rows:

Extra options

					name	average_rating
☐		Edit		Copy		Delete
					Avengers	4.75000
☐		Edit		Copy		Delete
					Inception	4.65000
☐		Edit		Copy		Delete
					Interstellar	4.75000

Extra options

AVG(rating)

4.33000